LAB

3

Operating Systems · FCIM / FAF · UTM · 2026-2027

The Command Line: Shell, Pipes and Text Filters

A **Track A: the real OS, from the outside (Linux) · 90 minutes**

AT A GLANCE

Part 1	Move fast, find help	~15 min
Part 2	What the shell does to your line	~15 min
Part 3	Redirection, chaining and jobs	~15 min
Part 4	Answer questions with one-liners	~25 min
Part 5	Your first script	~10 min
You submit	report.md: your predictions, outputs and answers for Parts 1 to 3, and the answers to Q1 to Q8.	

Goal

The terminal is the tool you will use every week of this course. Today you learn to move fast in bash, to predict what the shell does to your line before it runs it (globbing, quoting, expansions), to send output where you want it (redirection, chaining) and, above all, to answer real questions about data with one line of small filters joined by pipes.

Before you start

- Open a terminal on your Linux laptop. Keep report.md open beside it.
- If you have not prepared the laptop yet, run the course setup script once (it also prepares xv6 for Lab 4):

TERMINAL

```
bash os2026-setup.sh
```

- Generate the data set for Part 4 (an artificial web access log, the same for everyone):

TERMINAL

```
mkdir -p ~/os-lab3 && cd ~/os-lab3
for i in $(seq 1 500); do
  echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $((i % 5)) -eq 0 ] &&
echo FAIL || echo OK) /page$((i % 4))"
done > access.log
head -3 access.log ; wc -l access.log
```

The golden rule of this lab

Before you press Enter on any command marked "predict", write your prediction in report.md. Then run it. The mismatches are where you learn.

PART 1 Move fast, find help ~15 min

1.1 Type `ls /usr/sh` and press Tab twice. Then `cd /et` and Tab once. What did Tab do in each case?

1.2 Run three different commands, then press Ctrl+R and type a few letters of the first one. Press Enter.

1.3 Built-in or external? Where does the shell look?

TERMINAL

```
type cd ; type ls ; type echo ; which ls
echo $PATH
echo $PATH | tr ':' '\n'      # one directory per line
nosuchprog ; echo "exit code: $?"
```

1.4 Find help without a browser:

TERMINAL

```
man ls             # inside: /sort then n, q to quit
ls --help | head
man -k "list directory"
help cd
```

Observe (write in report.md)

1. Which of cd, ls, echo are built-ins, and why MUST cd be one?
2. How many directories are in your PATH, and in which one is ls?
3. What exit code does a command that is not found return?
4. Which option of ls sorts by modification time (find it in man ls)?

PART 2 What the shell does to your line ~15 min

TERMINAL

```
mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
ls
```

Predict the output of each line, write it down, then run it:

TERMINAL

```
echo *.c
echo file?.txt
echo [a-b]*
echo *.xyz
ls alfa beta.txt
ls "alfa beta.txt"
echo '$HOME' "$HOME" \$HOME
echo {1..5} ; echo $((7 * 6))
echo "today is $(date +%A), I am $(whoami)"
touch log{1..3}.txt && ls log*
```

Observe (write in report.md)

1. For each line: your prediction, the real output, and one sentence if they differ.
2. echo knows nothing about files, yet `echo *.c` printed file names. Who produced them?
3. Why did `ls alfa beta.txt` fail? Give two different ways to fix it.
4. What is the difference between single and double quotes?

PART 3 Redirection, chaining and jobs ~15 min

TERMINAL

```
cd ~/os-lab3
ls /etc /nope > out.txt 2> err.txt
wc -l out.txt ; cat err.txt
ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
echo first > notes.md ; echo second >> notes.md ; cat notes.md
wc -l < access.log
ls /nope 2> /dev/null || echo "failed with code $?"
mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
sleep 100 &
jobs ; kill %1 ; jobs
```

Observe (write in report.md)

1. Which file got the error message, and which got the listing?
2. What is the difference between `> all.txt 2>&1` and `2>&1 > all.txt`? Try both and explain.
3. Why does `wc -l < access.log` print only a number, while `wc -l access.log` also prints the name?
4. In one sentence each: ; && ||.

PART 4 Answer questions with one-liners ~25 min

Each line of access.log is: date, time, user, result (OK or FAIL), page. Answer every question with ONE command line (pipes allowed) and save each line in a file oneliners.sh, one per question, with the question as a comment above it. Useful tools: grep, cut, sort, uniq, wc, head, tail, tr.

- Q1. How many requests failed?
- Q2. How many different pages were requested, and which?
- Q3. How many requests did each page get?
- Q4. Which user or users have the most failed requests, and how many?
- Q5. How many requests did user3 make, and how many of them failed?
- Q6. Print the last 3 failed requests, showing only time and user.

Then two questions about the real system:

- Q7. Which login shells appear in /etc/passwd, and how many accounts use each? (field 7, separator :)
- Q8. Predict, then explain: why do `ls /etc | wc -l` and `ls -l /etc | wc -l` differ by exactly one?

TERMINAL

```
# a pattern to start from: count something per user
grep OK access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn
```

Check yourself

The log has 500 lines. If your Q3 counts do not add up to 500, or your Q4 counts do not add up to your Q1, something in the pipeline is filtering the wrong thing.

Stretch (optional, +1)

With grep -E, print only the lines where the minute ends in 0 (10:00, 10:10, ...) and the request failed. How many are there?

PART 5 Your first script ~10 min

Turn your best one-liners into a program. Create report.sh, make it executable, and run it on the log:

TERMINAL

```
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
# TODO: for each of user0 ... user6, print "userN: X failures"
```

TERMINAL

```
chmod +x report.sh
./report.sh access.log
./report.sh nope.log ; echo "exit code: $?"
```

Observe (write in report.md)

1. What does the first line #!/bin/bash do, and what happens if you run ./report.sh before chmod +x?
2. Why must you type ./report.sh and not just report.sh?
3. Why does the script write the error with >&2 and end with exit 1?

Stretch (optional, +1): run it with cron

Add a line to your crontab (crontab -e) that runs report.sh every minute and appends its output to ~/os-lab3/report.log (use full paths and >> ... 2>&1). Wait two minutes, show the log, then remove the line again (crontab -e, or crontab -r to delete the whole crontab). Paste your crontab line and the log in report.md.

FINAL Wrap-up and deliverable ~5 min

You submit:

- ☐ report.md: your predictions, outputs and answers for Parts 1 to 3, and the answers to Q1 to Q8.
- ☐ report.sh: your script from Part 5, with the TODO done.
- ☐ oneliners.sh: one commented command line per question. It must run with `bash oneliners.sh` inside ~/os-lab3.

Push both to your course repository; the assistant regenerates access.log and runs your one-liners.

Grading (out of 10, +2 bonus):

Criterion	Points
Part 1: built-ins, PATH, exit code, man	**1.5**
Part 2: predictions vs outputs, explained	**2**
Part 3: redirection and chaining explained	**2**
Part 4: Q1 to Q8 correct, one-liners run	**3.5**
Part 5: report.sh runs, loop over all users, errors handled	**1**
Bonus: the grep -E stretch	**+1**
Bonus: report.sh scheduled with cron, log shown	**+1**
Total	**10 (+2 bonus)**